JS Form Validation

K0126002 Web Programming

Gachon University · 2026 Fall · Dr. Toshpulatov

Why isn't HTML5 enough?

- Browser messages are ugly and not translatable
- No real-time feedback — only on submit

The novalidate Attribute

❌ **WITHOUT JS (browser controls everything)**

```
<!-- Browser handles everything — no JS control -->
<form action="/register" method="post">
  <input type="email" required>
  <button>Submit</button>
</form>
```

Without novalidate:

- Browser shows its own error bubbles
- Your JS errors also show
- User sees duplicate messages
- No CSS control over the tooltip

✅ **WITH JS (novalidate — JS takes over)**

```
<!-- novalidate disables browser UI — JS takes over -->
<form action="/register" method="post" novalidate>
  <input type="email" id="email" required
         aria-describedby="err-email">
  <span id="err-email" role="alert"></span>
  <button>Submit</button>
</form>
```

With novalidate:

- Browser stays silent — no tooltip bubbles
- Only your styled error appears
- checkValidity() still works normally
- Full CSS/design control over errors

preventDefault() + submit event

❌ **WITHOUT JS — form submits immediately**

```
<!-- Form submits immediately on button click -->
<form action="/register" method="post">
  <input type="text" name="name" required>
  <button type="submit">Register</button>
</form>
<!-- No JS = no validation = page reloads -->
```

✅ **WITH JS — preventDefault stops the reload**

```
const form = document.getElementById('regForm');

form.addEventListener('submit', function(event) {
  event.preventDefault(); // STOP the page reload

  if (validateAll()) {
    // All fields valid — safe to submit
    form.submit();
  } else {
    showErrors(); // Show what needs fixing
  }
});
```

Key concepts: submit event fires when form is submitted | **preventDefault()** stops the page from reloading | Without it, the browser sends the form BEFORE your JS validation runs.

Real-time Validation: the input Event

❌ **SUBMIT-ONLY — bad UX, user waits until submit**

```
form.addEventListener('submit', (e) => {
  // Only checked when user clicks Submit
  // Bad UX: user fills 10 fields, submits,
  // then sees the first field was wrong
  e.preventDefault();
  if (!sid.value.match(/^\d{9}$/)) {
    showError(sid, 'Must be 9 digits');
  }
});
```

✅ **REAL-TIME — feedback on every keystroke**

```
const sid = document.getElementById('sid');

// 'input' event fires on every keystroke
sid.addEventListener('input', function() {
  if (/^\d{9}$/.test(this.value)) {
    markValid(this);       // green border
  } else {
    markInvalid(this,      // red border
      'Must be exactly 9 digits');
  }
});
```

input event fires on every keystroke, paste, delete — enabling real-time feedback.
blur event fires when user leaves a field — good for "did they fill it?" **change event** fires after value changes and field loses focus.

Validation Checks: regex vs validity API

❌ **VAGUE — only checks empty, misses format**

```
// Only checks if field is empty — not format
if (!emailField.value) {
  showError('Email is required');
}
// Misses: "hello" (not an email format)
// Misses: "a@b" (incomplete domain)
```

`valueMissing` required + empty

`typeMismatch` email / url format wrong

`tooShort` minlength not met

`patternMismatch` pattern regex fails

`rangeUnderflow` below min= value

✅ **SPECIFIC — validity API + regex patterns**

```
// Approach 1: HTML5 validity API
const email = document.getElementById('v-email');
if (email.validity.valueMissing) {
  showError(email, 'Email is required');
} else if (email.validity.typeMismatch) {
  showError(email, 'Enter a valid email');
}

// Approach 2: Regex for custom patterns
const sid = document.getElementById('sid');
if (!/^\d{9}$/.test(sid.value)) {
  showError(sid, 'Must be exactly 9 digits');
}
```

QUICK QUIZ

What does event.preventDefault() do when called inside a form's submit handler?

A Deletes the form data

B Stops the form from reloading the page

C Validates all fields automatically

D Hides the form

Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

What does event.preventDefault() do when called inside a form's submit handler?

A Deletes the form data

B Stops the form from reloading the page

C Validates all fields automatically

D Hides the form

It cancels the default browser action (page reload + HTTP request), letting your JS handle submission instead. Without it, the page reloads before your JS validation code ever runs — so your checks are bypassed entirely.

Error Messages + ARIA Accessibility

❌ **WITHOUT ARIA — inaccessible**

```
<label for="sid">Student ID</label>
<input type="text" id="sid">
<!-- Error with no ARIA link — screen
     readers cannot find this message -->
<p class="err" id="err-sid">
  Must be 9 digits
</p>
```

`aria-describedby` links input to its error span

`aria-invalid="true"` tells AT the field has an error

`role="alert"` makes screen reader announce changes

✅ **WITH ARIA — screen reader friendly**

```
<label for="sid">Student ID</label>
<input type="text" id="sid"
  aria-describedby="err-sid"
  aria-invalid="true">
<span class="err" id="err-sid" role="alert">
  Must be exactly 9 digits
</span>
```

JS helper

```
function markInvalid(input, errSpan, msg) {
  input.setAttribute('aria-invalid','true');
  errSpan.textContent = msg;
  // role="alert" makes screen reader
  // announce the change immediately
}
```

⚡ QUICK QUIZ

You call field.setCustomValidity('') (empty string). What is the effect?

A Shows an error message

B Marks the field as valid / clears any custom error

C Deletes the field value

D Disables the field

💬 Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

You call field.setCustomValidity('') (empty string). What is the effect?

A Shows an error message

B Marks the field as valid / clears any custom error

C Deletes the field value

D Disables the field

An empty string resets the custom validity, making the field valid again. Any non-empty string sets an error message and marks the field invalid — checkValidity() returns false, and form.checkValidity() also returns false until it is cleared.

Password Confirmation with setCustomValidity()

❌ **HTML5 ALONE — no confirmation possible**

```
<!-- HTML5 has NO built-in password confirmation -->
<!-- This does NOT check that they match: -->
<input type="password" id="pw" required>
<input type="password" id="pw2" required>
<!-- The form will submit even if they differ! -->
```

Why setCustomValidity()?

It bridges your JS logic and the browser's validity API.

Non-empty string → field is invalid, checkValidity() = false

Empty string → field is valid, error cleared

✅ **WITH JS — setCustomValidity() comparison**

```
const pw  = document.getElementById('pw');
const pw2 = document.getElementById('pw2');

pw2.addEventListener('input', function() {
  if (this.value !== pw.value) {
    // Non-empty string = field is INVALID
    this.setCustomValidity(
      'Passwords do not match');
  } else {
    // Empty string = field is VALID again
    this.setCustomValidity('');
  }
});
```

⚡ QUICK QUIZ

Which event fires continuously AS the user types in an input field?

A keypress

B change

C input

D submit

💬 Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

Which event fires continuously AS the user types in an input field?

A keypress

B change

C input

D submit

The input event fires on every keystroke, enabling real-time feedback. change only fires when the field loses focus after the value changed. keypress is deprecated. submit only fires on form submit.

Disabling Submit Until Valid (Bonus)

❌ **NO JS — button always active**

```
<!-- Button is always clickable — no feedback -->
<button type="submit">Register</button>
<!-- User can click Submit even when
     required fields are empty -->
```

✅ **WITH JS — button disabled until form is valid**

```
// Check form validity on every keystroke
form.querySelectorAll('input, select')
  .forEach(el => {
    el.addEventListener('input', () => {
      // true only if ALL fields pass constraints
      submitBtn.disabled = !form.checkValidity();
    });
  });
```

`form.checkValidity()` returns true only when:

- All required fields are filled
- All pattern/type constraints pass
- No setCustomValidity() errors remain

Important: always validate on submit too — the disabled state can be bypassed in DevTools.

CSS for disabled state

```
button:disabled {
  opacity: 0.45;
  cursor: not-allowed;
  background: #94a3b8; /* gray */
}
button:enabled {
  background: #3b82f6; /* blue = ready */
}
```

QUICK QUIZ

The pattern /^\d{9}$/ validates a student ID. Which value PASSES this check?

A 12345678 (8 digits)

B 202312345 (9 digits)

C 20231234a (has a letter)

D 2023123456 (10 digits)

Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

The pattern /^\d{9}$/ validates a student ID. Which value PASSES this check?

A 12345678 (8 digits)

B 202312345 (9 digits)

C 20231234a (has a letter)

D 2023123456 (10 digits)

The regex requires exactly 9 digits: ^ = start, \d{9} = nine digits, $ = end. 202312345 has exactly 9 digits and passes. All others have wrong length or non-digit characters.

Full Pattern: Putting It Together

✅ **Part 1 — helpers + per-field validator**

```
const form      = document.getElementById('regForm');
const submitBtn = document.getElementById('submit-btn');

// 1. Reusable helper functions
function markValid(input, span) {
  input.setAttribute('aria-invalid', 'false');
  span.textContent = '';
}
function markInvalid(input, span, msg) {
  input.setAttribute('aria-invalid', 'true');
  span.textContent = msg;
}

// 2. Per-field validator
function validateEmail() {
  const el  = document.getElementById('v-email');
  const err = document.getElementById('err-email');
  if (el.validity.valueMissing)
    return markInvalid(el, err, 'Required'), false;
  if (el.validity.typeMismatch)
    return markInvalid(el, err, 'Invalid email'), false;
  return markValid(el, err), true;
}
```

✅ **Part 2 — real-time + submit wiring**

```
// 3. Real-time + submit wiring
document.getElementById('v-email')
  .addEventListener('input', () => {
    validateEmail();
    submitBtn.disabled = !form.checkValidity();
  });

form.addEventListener('submit', e => {
  e.preventDefault();
  if (validateEmail() /* && validateSid() */) {
    showSuccess();
  }
});

// Pattern summary:
// markValid / markInvalid  — helpers
// validateX()              — per field
// input listener           — real-time
// submit listener          — final gate
```

https://muxiddin19.github.io/WebPro/week7/JS_Form_Validation_Pattern.html

Summary + Homework

novalidate

Disables browser bubbles — JS takes over

preventDefault()

Stops form submit; run your checks first

input event

Real-time feedback on every keystroke

validity API

valueMissing, typeMismatch, tooShort…

setCustomValidity()

Custom errors + clear with ""

aria-describedby

Links input to error span for AT

HOMEWORK

Upgrade last class's registration form — add novalidate, preventDefault, input listeners for all fields, ARIA attributes (aria-describedby + aria-invalid), password confirmation with setCustomValidity(), and a success message div. Bonus: disable the Submit button until form.checkValidity() is true.

WebPro_StudyGuide_v2.html

WebPro_VideoLecture.html

WebPro_FormValidation_v3.pdf